

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO4: Examine intermediate code generation techniques to enhance code optimization (BL4)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:50

Annexure A

INDUSTRY PROBLEM- SOLVING TASK

Code of Conduct:

I D K Sumalatha certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

D K Sumalatha

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 25	/ 20	/ 20	/ 10	/ 100

Set 3

1. Autonomous Drone Delivery System

A logistics company is developing an autonomous drone delivery system to transport packages across urban areas. The compiler embedded in the drone controller processes declarations, assignment statements, and Boolean expressions to manage navigation, obstacle detection, and battery monitoring. During testing, inefficient intermediate code generation caused delays in route adjustment and emergency landing procedures. The organization plans to optimize the compiler for reliable real-time drone operations.

Tasks:

- (i) Design intermediate code for drone navigation and obstacle detection logic. (K3)
- (ii) Explain how Boolean expressions support autonomous decision-making in drones. (K4)
- (iii) Analyze the importance of compiler optimization in real-time delivery systems. (K5)

2. Railway Reservation and Scheduling System

A railway management company is modernizing its reservation and scheduling platform using a custom programming language. The compiler processes declarations, assignment statements, case statements, and procedure calls for seat allocation, train scheduling, and passenger status updates. During peak booking hours, improper intermediate code generation caused delays in reservation confirmation and ticket cancellation. The development team aims to redesign the compiler to improve scalability and transaction efficiency.

Tasks:

- (i) Construct intermediate code for reservation and scheduling operations. (K3)
- (ii) Discuss how case statements efficiently handle passenger ticket categories. (K4)
- (iii) Evaluate the role of compiler optimization in large-scale transportation systems. (K5)

3. Intelligent Energy Management System

An energy company is building a smart grid monitoring system that automatically controls electricity distribution based on power consumption. The compiler in the embedded controller evaluates Boolean expressions and assignment statements to monitor load balancing and fault detection. During high-demand periods, inefficient backpatching delayed fault recovery and power redistribution. The company plans to enhance compiler performance to improve reliability and energy efficiency.

Tasks:

- (i) Generate intermediate code for load balancing and fault detection logic. (K3)
- (ii) Explain the significance of backpatching in smart energy management systems. (K4)
- (iii) Analyze how compiler optimization improves the performance of smart grids. (K5)

4. AI-Based Fraud Detection System

A financial institution is developing an AI-based fraud detection platform that continuously analyzes transaction patterns. The compiler processes declarations, assignment statements, and Boolean conditions to identify suspicious activities and trigger alerts. During large-scale transaction processing, inefficient procedure calls increased response time and delayed fraud notifications. The software architects plan to redesign the compiler for faster analysis and improved scalability.

Tasks:

- (i) Develop intermediate code for fraud detection and alert generation logic. (K3)
- (ii) Explain how procedure calls improve modularity in fraud detection systems. (K4)
- (iii) Evaluate the role of compiler design in secure financial applications. (K5)

5. Smart Warehouse Inventory Management System

A retail company is implementing a smart warehouse inventory system that automatically tracks product movement and stock levels. The compiler processes assignment statements, Boolean expressions, and case statements to update inventory records and manage automated storage systems. During seasonal sales, delays in intermediate code execution affected stock synchronization and order fulfillment. The company plans to optimize the compiler to improve inventory accuracy and warehouse efficiency.

Tasks:

- (i) Design intermediate code for inventory tracking and stock update operations. (K3)
- (ii) Explain how case statements help manage multiple product categories efficiently. (K4)
- (iii) Analyze the importance of compiler optimization in warehouse automation systems. (K5)

1. Autonomous Drone Delivery System

(i) Intermediate code for drone navigation and obstacle detection (K3)

Source Logic

if obstacle = true then

land = true

else

move = nextRoute

Three Address Code (TAC)

1. if obstacle == true goto L1

2. move = nextRoute

3. goto L2

4. L1: land = true

5. L2: stop

Explanation: The drone checks for an obstacle. If found, it performs an emergency landing; otherwise, it follows the next route.

(ii) How Boolean expressions support autonomous decision-making (K4)

- Boolean expressions return True or False to help the drone make quick decisions.
- They are used for obstacle detection, battery checking, and choosing a safe flight path automatically.

(iii) Importance of compiler optimization in real-time delivery systems (K5)

- Optimization reduces execution time and improves route adjustment speed.
- It helps the drone respond quickly during emergencies and saves battery power.

2. Railway Reservation and Scheduling System

(i) Intermediate code for reservation and scheduling operations (K3)

Source Logic

seat = availableSeat

schedule = trainTime

call confirmReservation()

Three Address Code

1. seat = availableSeat

2. schedule = trainTime

3. param seat

4. param schedule

5. call confirmReservation

Explanation: The system assigns a seat, stores the train schedule, and calls the reservation confirmation procedure.

(ii) How case statements handle passenger ticket categories (K4)

case category of

Student: discount = 50

Senior: discount = 40

General: discount = 0

end

- Case statements select the correct ticket category directly.
- They reduce unnecessary comparisons and improve booking speed.

(iii) Role of compiler optimization in transportation systems (K5)

- Optimization speeds up reservation and cancellation during peak hours.

- It improves scalability by handling many passenger requests efficiently.

3. Intelligent Energy Management System

(i) Intermediate code for load balancing and fault detection (K3)

Source Logic

if load > limit then

redistribute = true

if fault == true then

powerOff = true

Three Address Code

1. if load > limit goto L1

2. goto L2

3. L1: redistribute = true

4. L2: if fault == true goto L3

5. goto L4

6. L3: powerOff = true

7. L4: stop

Explanation: The system balances the load when power exceeds the limit and turns off power if a fault is detected.

(ii) Significance of backpatching (K4)

- Backpatching fills the correct jump addresses after code generation.
- It ensures the system quickly reaches the correct recovery action during faults.

(iii) How compiler optimization improves smart grids (K5)

- It reduces processing delays during high-demand periods.
- Faster execution improves fault recovery and energy distribution reliability.

4. AI-Based Fraud Detection System

(i) Intermediate code for fraud detection and alert generation (K3)

Source Logic

if amount > limit then

call sendAlert()

Three Address Code

1. if amount > limit goto L1

2. goto L2
3. L1: call sendAlert
4. L2: stop

Explanation: The system checks whether the transaction exceeds the limit and sends an alert if it appears suspicious.

(ii) How procedure calls improve modularity (K4)

- Procedure calls separate tasks such as transaction checking and alert generation.
- They make the program easier to maintain, update, and reuse.

(iii) Role of compiler design in secure financial applications (K5)

- Efficient compiler design speeds up fraud detection and alert generation.
- It improves security, reliability, and performance when processing large numbers of transactions.

5. Smart Warehouse Inventory Management System

(i) Intermediate code for inventory tracking and stock updates (K3)

Source Logic

stock = stock - sold

if stock < minimum then

reorder = true

Three Address Code

1. stock = stock - sold

2. if stock < minimum goto L1

3. goto L2

4. L1: reorder = true

5. L2: stop

Explanation: The system updates the stock after sales and automatically starts the reorder process when stock becomes low.

(ii) How case statements manage multiple product categories (K4)

case product of

Electronics: rack = A

Clothing: rack = B

Grocery: rack = C

end

- Case statements quickly select the correct storage location for each product category.
- They simplify inventory management and reduce processing time.

(iii) Importance of compiler optimization in warehouse automation (K5)

- Optimization speeds up inventory updates and order processing.
- It improves stock accuracy and helps automated systems work more efficiently.

1. Autonomous Drone Delivery System

(i) Design intermediate code for drone navigation and obstacle detection logic. (K3)

Source Program

```
if (obstacle == true)
```

```
land = true;
```

```
else
```

```
move = nextRoute;
```

Three Address Code (TAC)

1. t1 = obstacle == true

2. if t1 goto L1

3. move = nextRoute

4. goto L2

5. L1: land = true

6. L2: stop

Quadruple Representation

Op	Arg1	Arg2	Result
==	obstacle	true	t1
if	t1	—	L1
=	nextRoute	—	move
goto	—	—	L2
=	true	—	land

Explanation:

- The compiler first checks whether an obstacle exists.
- If an obstacle is detected, the drone lands immediately.

- Otherwise, it continues to the next route.

(ii) Explain how Boolean expressions support autonomous decision-making in drones. (K4)

Boolean expressions evaluate conditions as True or False. They allow the drone to make automatic decisions without human control.

Examples:

- `battery < 20` → Return to charging station.
- `obstacle == true` → Emergency landing.
- `GPS_signal == false` → Hover until the signal returns.
- `windSpeed > safeLimit` → Cancel the flight.

Advantages

- Fast decision-making.
- Improves flight safety.
- Reduces manual intervention.
- Supports real-time navigation.

(iii) Analyze the importance of compiler optimization in real-time delivery systems. (K5)

Compiler optimization improves the efficiency of the generated machine code.

Importance

- Reduces execution time during route calculations.
- Makes emergency landings faster.
- Saves battery by reducing unnecessary instructions.
- Improves obstacle response time.
- Ensures smooth real-time operation.

Without optimization

- Slow route updates.
- Delayed emergency responses.
- Increased power consumption.

2. Railway Reservation and Scheduling System

(i) Construct intermediate code for reservation and scheduling operations. (K3)

Source Program

```
seat = availableSeat;
```



```
schedule = trainTime;  
confirmReservation(seat, schedule);
```

Three Address Code

1. seat = availableSeat
2. schedule = trainTime
3. param seat
4. param schedule
5. call confirmReservation,2

Quadruple Representation

Op	Arg1	Arg2	Result
=	availableSeat	—	seat
=	trainTime	—	schedule
param	seat	—	—
param	schedule	—	—
call	confirmReservation	2	—

Explanation

- The compiler assigns a seat.
- Stores the train schedule.
- Passes both values to the reservation procedure.

(ii) Discuss how case statements efficiently handle passenger ticket categories. (K4)

Example

case category of

Student : discount = 50;

Senior : discount = 40;

Child : discount = 60;

General : discount = 0;

end

Why it is efficient

- Directly selects the correct category.
- Avoids multiple nested if-else statements.

- Makes the code easier to read.
- Increases booking speed during heavy traffic.

(iii) Evaluate the role of compiler optimization in large-scale transportation systems. (K5)

A railway system processes thousands of bookings every minute.

Benefits

- Faster reservation confirmation.
- Quick ticket cancellation.
- Efficient train scheduling.
- Better memory usage.
- Supports many users simultaneously.

Result

- Reduced waiting time.
- Improved passenger experience.
- Better system reliability.

3. Intelligent Energy Management System

(i) Generate intermediate code for load balancing and fault detection logic. (K3)

Source Program

```
if(load > limit)
    redistribute = true;
```

```
if(fault == true)
```

```
    powerOff = true;
```

Three Address Code

1. t1 = load > limit
2. if t1 goto L1
3. goto L2
4. L1: redistribute = true
5. L2: t2 = fault == true
6. if t2 goto L3
7. goto L4

8. L3: powerOff = true

9. L4: stop

Explanation

- The system checks whether the electrical load exceeds the limit.
- If yes, it redistributes power.
- Then it checks for faults and turns off power if necessary.

(ii) Explain the significance of backpatching in smart energy management systems. (K4)

Backpatching fills unknown jump addresses after generating intermediate code.

Example

Before:

if fault goto ____

After backpatching:

if fault goto L3

Importance

- Corrects jump targets.
- Enables quick fault recovery.
- Improves load balancing.
- Makes control flow accurate.

(iii) Analyze how compiler optimization improves smart grids. (K5)

Optimization helps the smart grid respond quickly.

Benefits

- Faster fault detection.
- Reduced recovery time.
- Better electricity distribution.
- Lower processor workload.
- Increased energy efficiency.

Result

- Reliable power supply.
- Fewer interruptions.
- Improved system performance.

4. AI-Based Fraud Detection System

(i) Develop intermediate code for fraud detection and alert generation logic. (K3)

Source Program

```
if(amount > limit)
```

```
sendAlert();
```

Three Address Code

1. t1 = amount > limit

2. if t1 goto L1

3. goto L2

4. L1: call sendAlert,0

5. L2: stop

Quadruple Representation

Op	Arg1	Arg2	Result
>	amount	limit	t1
if	t1	—	L1
call	sendAlert	0	—

Explanation

- The compiler compares the transaction amount with the allowed limit.
- If suspicious, it immediately calls the alert procedure.

(ii) Explain how procedure calls improve modularity in fraud detection systems. (K4)

A procedure call allows each task to be written separately.

Example:

```
checkTransaction()
```

```
sendAlert()
```

```
updateLog()
```

Advantages

- Reusable code.
- Easier debugging.
- Better maintenance.
- Faster development.

(iii) Evaluate the role of compiler design in secure financial applications. (K5)

Good compiler design improves both speed and security.

Benefits

- Faster transaction processing.
- Quick fraud detection.
- Efficient memory management.
- Reliable alert generation.
- Better scalability for millions of users.

Result

- Secure banking operations.
- Reduced financial risk.
- Improved customer trust.

5. Smart Warehouse Inventory Management System

(i) Design intermediate code for inventory tracking and stock update operations. (K3)

Source Program

stock = stock - sold;

if(stock < minimum)

reorder = true;

Three Address Code

1. t1 = stock - sold

2. stock = t1

3. t2 = stock < minimum

4. if t2 goto L1

5. goto L2

6. L1: reorder = true

7. L2: stop

Quadruple Representation

Op	Arg1	Arg2	Result
-	stock	sold	t1
=	t1	—	stock

<	stock	minimum	t2
if	t2	—	L1
=	true	—	reorder

Explanation

- Stock is reduced after every sale.
- When stock becomes lower than the minimum level, the reorder process starts automatically.

(ii) Explain how case statements help manage multiple product categories efficiently. (K4)

Example

case product of

Electronics : rack = A;

Clothing : rack = B;

Grocery : rack = C;

Furniture : rack = D;

end

Benefits

- Quickly selects the correct storage rack.
- Reduces unnecessary condition checking.
- Makes warehouse operations faster.
- Improves inventory organization.

(iii) Analyze the importance of compiler optimization in warehouse automation systems. (K5)

Compiler optimization helps warehouse software run efficiently.

Advantages

- Faster stock updates.
- Quick order fulfillment.
- Better synchronization between automated robots and inventory records.
- Reduced memory and CPU usage.
- Improved warehouse productivity.

Overall Result

- Accurate inventory records.
- Faster deliveries.
- Better customer satisfaction.
- Efficient warehouse automation.